

Rendu Personnel

SAE 3.02 Développer des
applications communicantes



Sommaire

Table des matières

Sommaire	1
Présentation du projet	3
Nom & description :	3
Mais en quoi consiste-t-il ?	3
Répartition du travail en groupe	3
Constitution du groupe : Emeraldas :	3
Tâches à réaliser :	4
Planning de développement	5
Durée totale du projet et répartition des tâches	5
Jalons clés	5
Organisation et outils	6
Outils :	6
Risques identifiés	6
Création des VMs	7
Introduction :	7
Développement de l'installation :	7
Création du code pour le Master	9
Recherches	9
Évolution du code	9
Les bibliothèques utilisées	9
La première classe « Routeur »	10
-> def _init_	10
-> def vers_texte	10
La deuxième classe « ServeurMaster »	10
-> def _init_	10
-> def demarrer	11
-> def lire_ligne	12
-> def envoyer_ligne	12
-> def traiter_commande	12
-> def enregistrer_routeur	13
-> def generer_chemin	13
-> _main_	14
Création du code pour le(s) Client(s)	15

Recherches	15
Évolution du code	15
Les bibliothèques utilisées	15
La classe « TerminalClient »	16
-> def __init__	16
-> def envoyer_ligne	16
-> def lire_ligne	16
-> def attendre_messages	17
-> traiter_connexion	17
-> demander_chemin	18
-> fabriquer_message	18
-> def envoyer	19
-> def lancer	19
-> __main__	20
Création du code pour le(s) Routeur(s)	21
Recherches	21
Évolution du code	21
Les bibliothèques utilisées	21
La classe « NoeudIntermédiaire »	22
-> def __init__	22
-> def ecrire_ligne	22
-> def lire_ligne	22
-> def inscription_master	23
-> def boucle_ecoute	23
-> def traiter_connexion	24
-> def transmettre	25
-> def demarrer	25
-> __main__	26
Compétences Acquises	27
Conclusion	28

Présentation du projet

Nom & description :

Le projet se nomme « Conception d'une architecture distribuée avec routage en oignon »

Mais en quoi consiste-t-il ?

Ce projet consiste à concevoir et implémenter un système de communication anonyme basé sur une architecture distribuée et un routage en oignon.

Le système repose sur plusieurs routeurs virtuels, un serveur maître et des clients capables d'échanger des messages de manière anonyme.

L'objectif principal est de garantir la confidentialité et l'anonymat des utilisateurs grâce à un chiffrement asymétrique simplifié et une architecture multi-sauts.

Le public cible est composé des étudiants, des enseignants ou tout autre utilisateur souhaitant analyser, comprendre ou démontrer le fonctionnement de cette architecture.

Répartition du travail en groupe

Constitution du groupe : Emeraldas :

⇒ *Imène MIRA & Chloé BRISSE*

Prenant en compte le fait que nous sommes uniquement deux dans le groupe, nous ne voyons pas la nécessité de désigner un chef de projet.

Pour une meilleure efficacité, nous préférons partager toutes les tâches. En effet, les tâches sont réparties mais vérifiées par le groupe

Tâches à réaliser :

- ⇒ Développement du programme de chiffrement (inclus la modification du programme master pour rajouter le chiffrement)
 - BRISSE Chloé
- ⇒ Mise en place de la base de données MariaDB.
 - Tâche partagée
- ⇒ Développement des programmes routeurs, client et master.
 - MIRA Imène
- ⇒ Gestion du dépôt Git et suivi de projet.
 - Tâche partagée
- ⇒ Conception du planning et documentation (Document de suivi mis à jour tout au long du projet)
 - Tâche partagée
- ⇒ Création des VMs avec la configuration adapté
 - MIRA Imène

Planning de développement

Durée totale du projet et répartition des tâches

Le Gantt est disponible sur GitHub. Onglet « Project ».

Pour nous, le projet s'étend sur toute la durée disponible, c'est-à-dire jusqu'au 31 décembre. Nous ne voulons pas nous dire inconsciemment "il y a de la marge" et risquer, au final, d'avoir du retard. Nous visons de finir en avance le travail avant le 31 décembre.

Jalons clés

Les tâches sont réparties au fur et à mesure de l'évolution du projet, s'il y a du retard, si l'une de nous a plus d'affinité ou pour toutes autres raisons. Nous préférons être flexibles pour éviter de rester bloqués sur une tâche, il nous est même possible de travailler en même temps sur un élément ou de refaire une répartition des tâches si l'une de nous est dans une impasse. Il s'agit de travailler de concert et de travailler de façon fusionnelle pour rendre le meilleur travail possible.

Organisation et outils

Outils :

Suivi projet : GitHub & GitHub Project,

Base de données : MariaDB,

Communication : Discord, WhatsApp, Messagerie, Mail UHA

Gestion de projet : GitHub

Développement : Python, PyQt5/6, Socket, Threading.

Risques identifiés

- ⇒ **Complexité technique** : gestion correcte des threads et sockets → test progressif, opérationnalité du code de chiffrement RSA.
- ⇒ **Dépendances externes** : MariaDB → prévoir un script d'installation,
- ⇒ **Débogage multi-machines** : variabilité des environnements → documentation précise,
- ⇒ **Temps** : charge importante → planning strict et jalons obligatoires.

Création des VMs

Introduction :

Le projet consistait à faire communiquer le Routeur, le Client et le Master en utilisant trois Machines Virtuelles.

- ✓ La Client est sur une machine virtuelle Windows,
- ✓ Le Routeur est sur une machine virtuelle Linux,
- ✓ Le Master est sur une machine virtuelle Linux.

Développement de l'installation :

Création du réseau Host-Only (pour faire un test en local)

- ❖ Ouvrir VirtualBox
- ❖ Menu Fichier → Outils → Gestionnaire de réseau hôte
- ❖ Cliquez sur Créer
- ❖ Réseau : vboxnet0
- ❖ IP hôte : 192.168.56.1
- ❖ DHCP : activé ou désactivé (fortement déconseillé pour ce projet)
- ❖ relie VM ↔ VM
- ❖ relie VM ↔ machine hôte

Configuration des VMs Linux

(Installer avec un ISO la VM Linux - ISO disponible sur internet)

Configuration puis dans Réseau (identique pour les deux VMs - Master & Routeur)

- ❖ Carte 1 = mode accès réseau = Réseau privé hôte
- ❖ Nom : vboxnet0

Configuration de la VM Windows

(Installer avec un ISO la VM Linux - ISO disponible sur internet)

Configuration puis dans Réseau

- ❖ Carte 1 = mode accès réseau = Réseau privé hôte
- ❖ Nom : vboxnet0

Afin de vérifier que tout est correctement configuré, il est possible de :

Dans Linux :

```
>> ip a  
#permet d'afficher toute la configuration réseau de la VM
```

Dans Windows :

```
>> ipconfig (ou ipconfig /all pour plus de détails)  
#permet d'afficher toute la configuration réseau de la VM
```

- **Puis, pour plus de vérifications, il est possible de faire des « ping » entre les différentes machines (en utilisant l'adresse IP qui à été configurées sur chaque VM)**
- **Attention : lorsque les programmes seront dans les VMs, il sera impératif de modifier les IP qui y sont écrites, autrement, ça ne fonctionnera pas...**

Création du code pour le Master

Recherches

Pour comprendre le fonctionnement entre les différents éléments de la maquette, il m'a fallu réaliser quelques recherches sur internet, notamment sur Wikipédia ainsi que d'autres sites permettant de mettre en place les différentes parties du programme.

Évolution du code

Tout d'abord, nous avons réalisé un schéma afin de mieux comprendre le fonctionnement. Après plusieurs croquis et brouillons des différents codes, j'ai pu réaliser un premier lien entre le master, le routeur et le client.

Une fois le lien établi, j'ai réalisé deux autres routeurs pour au total en avoir minimum 3 puis un autre client pour en avoir minimum deux. Durant la réalisation du programme, chaque ligne a été expliquée par des « Docstring »

Les bibliothèques utilisées

Dans trois bibliothèques python ont été utilisées, la bibliothèque « `socket` » qui permet de réaliser la communication réseau entre les différents programmes, la bibliothèque « `threading` » qui permet de gérer plusieurs connexions en parallèle et la bibliothèque « `random` » permet de choisir des routeurs aléatoirement pour créer des chemins variés (pour un premier test)

```
import socket          # Module pour la communication réseau (TCP)
import threading        # Module pour gérer plusieurs connexions en parallèle
import random           # Module pour la sélection aléatoire
```

La première classe « Routeur »

-> `def __init__`

Ce constructeur initialise un objet routeur en stockant son identifiant, son adresse IP et son port. Ces informations permettent au serveur master d'identifier le routeur et de savoir comment le contacter lorsqu'il est sélectionné dans un chemin.

-> `def vers_texte`

Cette méthode transforme les informations du routeur en une chaîne de caractères formatée. Ce format texte est utilisé par le master pour transmettre facilement les informations du routeur aux clients lors de la génération d'un chemin.

```
"""
*****
Classe Routeur
*****
"""

class Routeur:                                # Classe représentant un routeur enregistré

    """Représente un routeur connu par le serveur master"""

    def __init__(self, identifiant: str, ip: str, port: int): # Constructeur du routeur
        self.identifiant = identifiant                    # Identifiant unique du routeur
        self.ip = ip                                      # Adresse IP du routeur
        self.port = port                                  # Port utilisé par le routeur

    def vers_texte(self) -> str:                        # Convertit le routeur en chaîne de caractères
        return f"{self.identifiant},{self.ip},{self.port}" # Format texte du routeur
```

La deuxième classe « ServeurMaster »

-> `def __init__`

Cette méthode initialise le serveur master. Elle définit l'adresse IP et le port d'écoute, crée la liste qui contiendra les routeurs enregistrés, met en place un verrou pour sécuriser l'accès concurrent à cette liste et crée le socket réseau principal du serveur.

```
"""
*****
Classe Serveur/Master
*****
"""

class ServeurMaster:                            # Classe principale du serveur master
    def __init__(self, adresse=ADRESSE_SERVEUR, port=PORT_SERVEUR): # Constructeur
        self.adresse = adresse                    # Adresse IP du serveur
        self.port = port                          # Port du serveur

        self.routeurs = []                        # Liste des routeurs enregistrés
        self.verrou_routeurs = threading.Lock()  # Verrou pour protéger l'accès concurrent

        self.socket_serveur = socket.socket(     # Création du socket serveur
            socket.AF_INET,                       # Utilisation d'IPv4
            socket.SOCK_STREAM                    # Utilisation du protocole TCP
```

```
)
```

-> **def demarrer**

Cette méthode lance le serveur master. Elle associe le socket à l'adresse IP et au port, met le serveur en attente de connexions et accepte les connexions entrantes en continu. Chaque connexion est traitée dans un thread distinct afin de permettre plusieurs communications simultanées.

```
def demarrer(self):
    # Démarre le serveur master
    self.socket_serveur.bind((self.adresse, self.port)) # Association IP / port
    self.socket_serveur.listen(10) # Autorise jusqu'à 10 connexions simultanées
    print(f"[MASTER] Serveur démarré sur {self.adresse}:{self.port}") # Message d'état

    try:
        while True:
            # Boucle infinie d'attente de connexions
            socket_client, adresse_client = self.socket_serveur.accept() # Accepte un client
            print(f"[MASTER] Connexion reçue depuis {adresse_client}") # Log connexion

            threading.Thread(
                target=self.gestion_client, # Fonction exécutée dans le thread
                args=(socket_client, adresse_client), # Paramètres
                daemon=True # Thread lié au programme principal
            ).start() # Démarrage du thread

    finally:
        self.socket_serveur.close() # Fermeture du socket serveur
```

-> **def gestion_client**

Cette méthode gère une connexion individuelle avec un client ou un routeur. Elle lit le message reçu, le traite, envoie une réponse si nécessaire et ferme la connexion proprement. Elle sert de point central pour toutes les communications avec le master.

```
def gestion_client(self, socket_client: socket.socket, adresse_client): # Gère un client
    try:
        message = self.lire_ligne(socket_client) # Lecture du message du client

        if not message:
            # Si aucun message reçu
            return # Fin de la session

        print(f"[MASTER] Message reçu ({adresse_client}) : {message}") # Affichage

        reponse = self.traiter_commande(message) # Analyse de la commande

        if reponse:
            # Si une réponse existe
            self.envoyer_ligne(socket_client, reponse) # Envoi au client

    except Exception as erreur:
        # Gestion des erreurs
        print(f"[MASTER][ERREUR] {adresse_client} -> {erreur}") # Log erreur

    finally:
        socket_client.close() # Fermeture de la connexion
        print(f"[MASTER] Connexion fermée avec {adresse_client}") # Log fermeture
```

-> **def lire_ligne**

Cette méthode lit un message complet envoyé sur un socket, octet par octet, jusqu'à détecter un saut de ligne. Elle garantit que le message est reçu entièrement avant d'être analysé, puis le convertit en chaîne de caractères exploitable.

```
def lire_ligne(self, sock: socket.socket) -> str | None: # Lecture ligne par ligne
    tampon = bytearray() # Tampon de réception

    while True:
        octet = sock.recv(1) # Lecture d'un octet

        if not octet: # Si la connexion est fermée
            return None if not tampon else tampon.decode() # Retour des données

        if octet == b'\n': # Fin de ligne détectée
            break # Sortie de la boucle

        tampon.extend(octet) # Ajout de l'octet au tampon

    return tampon.decode() # Conversion en chaîne UTF-8
```

-> **def envoyer_ligne**

Cette méthode envoie un message texte à travers un socket réseau. Elle ajoute automatiquement un saut de ligne à la fin du message afin de respecter le protocole de communication utilisé entre les entités.

```
def envoyer_ligne(self, sock: socket.socket, texte: str): # Envoi d'une ligne
    sock.sendall((texte + "\n").encode()) # Envoi du message avec saut de ligne
```

-> **def traiter_commande**

Cette méthode analyse le message reçu par le serveur master. Elle découpe le message, identifie la commande demandée et appelle la méthode correspondante. Si la commande est inconnue ou invalide, un message d'erreur est retourné

```
def traiter_commande(self, ligne: str) -> str: # Analyse de la commande reçue
    morceaux = ligne.strip().split(";") # Découpage selon le protocole

    if not morceaux: # Message vide
        return "ERROR;MESSAGE_VIDE" # Erreur

    commande = morceaux[0].upper() # Commande principale

    match commande: # Analyse de la commande
        case "ROUTER_REGISTER": # Enregistrement d'un routeur
            return self.enregistrer_routeur(morceaux)

        case "CLIENT_GET_PATH": # Demande de chemin
            return self.generer_chemin(morceaux)

        case _: # Commande inconnue
            return "ERROR;COMMANDE_INCONNUE"

    )
```

-> **def enregistrer_routeur**

Cette méthode gère l'enregistrement des routeurs auprès du master. Elle vérifie la validité des informations reçues, ajoute un nouveau routeur à la liste ou met à jour ses données s'il existe déjà. Elle permet au master de maintenir une liste cohérente et à jour des routeurs disponibles.

```
def enregistrer_routeur(self, donnees: list[str]) -> str: # Ajout / mise à jour routeur
    if len(donnees) != 4:
        return "ERROR;FORMAT_ROUTEUR" # Vérification du format

    _, identifiant, ip, port_texte = donnees # Extraction des champs

    try:
        port = int(port_texte) # Conversion du port
    except ValueError:
        return "ERROR;PORT_INVALIDE" # Erreur si conversion impossible

    with self.verrou_routeurs: # Accès sécurisé à la liste
        for routeur in self.routeurs: # Recherche d'un routeur existant
            if routeur.identifiant == identifiant:
                routeur.ip = ip # Mise à jour IP
                routeur.port = port # Mise à jour port
                return f"OK;REGISTERED;{identifiant}"

        self.routeurs.append(Routeur(identifiant, ip, port)) # Ajout nouveau routeur

    return f"OK;REGISTERED;{identifiant}" # Confirmation
```

-> **def generer_chemin**

Cette méthode traite les demandes de chemin envoyées par les clients. Elle vérifie que le nombre de routeurs demandé est valide, sélectionne aléatoirement les routeurs nécessaires parmi ceux enregistrés et construit une réponse structurée contenant le chemin de routage.

```
def generer_chemin(self, donnees: list[str]) -> str: # Génération du chemin
    if len(donnees) != 2:
        return "ERROR;FORMAT_CHEMIN" # Vérification du format

    try:
        nombre_sauts = int(donnees[1]) # Nombre de routeurs demandés
        if nombre_sauts <= 0:
            raise ValueError
    except ValueError:
        return "ERROR;NB_SAUTS_INVALIDE" # Valeur incorrecte

    with self.verrou_routeurs: # Accès sécurisé
        if len(self.routeurs) < nombre_sauts:
            return "ERROR;PAS_ASSEZ_DE_ROUTEURS"

        selection = random.sample(self.routeurs, nombre_sauts) # Sélection aléatoire

    chemin = "|".join(r.vers_texte() for r in selection) # Construction du chemin
    return f"PATH;{chemin}" # Retour au client
```

-> `__main__`

Cette partie constitue le point d'entrée du programme. Elle crée une instance du serveur master et démarre son fonctionnement. Sans cette section, le serveur ne s'exécuterait pas automatiquement.

```
"""
*****
Lancement programmes
*****
"""

if __name__ == "__main__":
    serveur = ServeurMaster()      # Point d'entrée du programme
    serveur.démarrer()             # Création du serveur master
                                   # Démarrage du serveur

#python3 masterX.py = permet de lancer le master en fonction du choix du programme
```

Création du code pour le(s) Client(s)

Recherches

Pour comprendre le fonctionnement entre les différents éléments de la maquette, il m'a fallu réaliser quelque recherches sur internet, notamment sur Wikipédia ainsi que d'autres sites permettant de mettre en place les différentes parties du programme.

Évolution du code

Tout d'abord, nous avons réalisé un schéma afin de mieux comprendre le fonctionnement. Après plusieurs croquis et brouillons des différents codes, j'ai pu réalisé un premier lien entre le master, le routeur et le client.

Une fois le lien établi, j'ai réalisé deux autres routeurs pour au total en avoir minimum 3 puis un autre client pour en avoir minimum deux. Durant la réalisation des programme, chaque lignes à été expliqué par des « Docstring »

Les bibliothèques utilisées

Dans deux bibliothèques python ont été utilisées , la bibliothèque « `socket` » qui permet de réaliser la communication réseau entre les différents programmes et la bibliothèque « `threading` » qui permet de gérer plusieurs connexions en parallèle

```
import socket          # Module pour la communication réseau (TCP)
import threading       # Module pour gérer plusieurs connexions en parallèle
```


La classe « TerminalClient »

-> `def __init__`

Cette méthode initialise le client. Elle définit son nom, son adresse IP, son port d'écoute ainsi que l'adresse et le port du serveur master. Elle enregistre également le nombre de routeurs à utiliser par défaut. Enfin, elle crée et configure un socket serveur local qui permettra au client de recevoir des messages entrant

```
def __init__(self, nom, ip, port, master_ip, master_port, sauts_defaut): # Constructeur
    self.nom = nom # Nom du client (ex: A ou B)
    self.ip = ip # Adresse IP du client
    self.port = port # Port d'écoute du client
    self.master_ip = master_ip # Adresse IP du master
    self.master_port = master_port # Port du master
    self.sauts_defaut = sauts_defaut # Nombre de routeurs par défaut

    self.socket_ecoute = socket.socket(socket.AF_INET, socket.SOCK_STREAM) # Création du socket TCP
    self.socket_ecoute.bind((self.ip, self.port)) # Association IP/port
    self.socket_ecoute.listen(1) # Mise en écoute du
socket
```

-> `def envoyer_ligne`

Cette méthode permet d'envoyer un message texte sur un socket. Elle s'assure que le message se termine par un saut de ligne afin de respecter le protocole de communication, puis encode et transmet les données sur le réseau.

```
# -----
# Outils pour les sockets
# -----

def envoyer_ligne(self, s, contenu): # Fonction pour envoyer une ligne texte
    data = contenu if contenu.endswith("\n") else contenu + "\n" # Ajoute \n si absent
    s.sendall(data.encode()) # Envoi des données encodées en UTF-8
```

-> `def lire_ligne`

Cette méthode lit un message complet depuis un socket réseau. Elle reçoit les données octet par octet jusqu'à détecter un saut de ligne ou une fermeture de connexion. Le message reçu est ensuite reconstruit sous forme de chaîne de caractères et retourné.

```
def lire_ligne(self, s): # Fonction pour lire une ligne complète
    resultat = "" # Chaîne de caractères reçue
    fini = False # Indicateur de fin de ligne
    while not fini: # Boucle jusqu'à réception complète
        octet = s.recv(1) # Lecture d'un octet depuis le socket
        if octet == b"": # Si la connexion est fermée
            break # Sort de la boucle
        if octet == b"\n": # Si fin de ligne détectée
            fini = True # On marque la fin
        else:
            resultat += octet.decode() # Ajoute l'octet décodé à la chaîne
    return resultat if resultat else None # Retourne le message ou None
```

-> **def attendre_messages**

Cette méthode transforme le client en serveur local. Elle met le client en attente de connexions entrantes, généralement en provenance d'un routeur. Chaque nouvelle connexion est traitée dans un thread séparé afin de permettre la réception simultanée de plusieurs messages.

```
# -----
# Partie Serveur - Client
# -----

def attendre_messages(self):          # Serveur local du client
    print(f"[{self.nom}] Client actif sur {self.ip}:{self.port}") # Message d'état

    actif = True                      # Variable de contrôle
    while actif:                      # Boucle d'écoute permanente
        connexion, adresse = self.socket_ecoute.accept() # Acceptation d'une connexion
        t = threading.Thread(        # Création d'un thread
            target=self.traiter_connexion, # Fonction exécutée par le thread
            args=(connexion, adresse)     # Arguments transmis
        )
        t.daemon = True              # Thread en mode daemon
        t.start()                    # Démarrage du thread
```

-> **traiter_connexion**

Cette méthode gère une connexion entrante unique. Elle lit le message reçu, ferme la connexion et affiche le contenu du message si celui-ci est valide. Elle permet au client de recevoir les messages finaux envoyés par les routeurs.

```
def traiter_connexion(self, s, origine): # Traitement d'un message entrant
    contenu = None                       # Variable pour stocker le message
    try:
        contenu = self.lire_ligne(s)     # Lecture du message reçu
    except:
        pass                             # Ignore les erreurs
    finally:
        s.close()                       # Fermeture du socket

    if contenu:                           # Si un message valide est reçu
        print(f"[{self.nom}] Message reçu : {contenu}") # Affichage
```

-> demander_chemin

Cette méthode contacte le serveur master afin de demander un chemin de routage composé d'un certain nombre de routeurs. Elle envoie une requête au master, lit la réponse et analyse les informations reçues pour construire une liste ordonnée de routeurs constituant le chemin.

```
def demander_chemin(self, sauts):      # Demande d'un chemin au master
    socket_tmp = socket.socket(socket.AF_INET, socket.SOCK_STREAM) # Socket temporaire

    try:
        socket_tmp.connect((self.master_ip, self.master_port))      # Connexion au master
        self.envoyer_ligne(socket_tmp, f"CLIENT_GET_PATH;{sauts}") # Envoi de la requête

        reponse = self.lire_ligne(socket_tmp) # Lecture de la réponse
    finally:
        socket_tmp.close()      # Fermeture du socket

    if not reponse or "PATH;" not in reponse: # Vérification de la réponse
        return []                  # Retourne une liste vide si erreur

    chaine = reponse.split("PATH;", 1)[1] # Extraction du contenu utile
    blocs = chaine.split("|")             # Séparation des routeurs

    chemin = []                          # Liste finale du chemin
    index = 0                            # Index manuel
    while index < len(blocs):             # Boucle de traitement
        elements = blocs[index].split(",") # Découpage des infos routeur
        chemin.append((elements[0], elements[1], int(elements[2]))) # Ajout au chemin
        index += 1                       # Incrémentation de l'index

    return chemin                        # Retourne le chemin complet
```

-> fabriquer_message

Cette méthode construit le message à transmettre à travers les routeurs. Elle commence par créer le message final destiné au client de destination, puis encapsule ce message dans plusieurs couches correspondant aux routeurs intermédiaires, en respectant l'ordre inverse du chemin.

```
# -----
# Construction du message
# -----

def fabriquer_message(self, chemin, ip_final, port_final, texte): # Construction du message
    enveloppe = f"DELIVER;{ip_final};{port_final};{texte}"        # Message final de base

    chemin_inverse = chemin[:]      # Copie du chemin
    chemin_inverse.reverse()         # Inversion du chemin

    for info in chemin_inverse[:-1]: # Parcours sauf le premier routeur
        enveloppe = f"NEXT;{info[1]};{info[2]};{enveloppe}" # Encapsulation

    return enveloppe                # Retourne le message final
```

-> **def envoyer**

Cette méthode gère l'envoi complet d'un message. Elle demande d'abord un chemin au master, construit ensuite le message encapsulé à partir de ce chemin, puis envoie ce message au premier routeur du parcours. Elle assure également la gestion des erreurs de connexion.

```
# -----
# Envoi final
# -----

def envoyer(self, ip_dest, port_dest, texte, sauts=None): # Fonction d'envoi
    if sauts is None: # Si aucun nombre de sauts fourni
        sauts = self.sauts_default # Utilise la valeur par défaut

    route = self.demander_chemin(sauts) # Demande du chemin
    if len(route) == 0: # Si aucun routeur disponible
        print("[CLIENT] Aucun routeur disponible") # Message d'erreur
        return # Arrêt de la fonction

    paquet = self.fabriquer_message(route, ip_dest, port_dest, texte) # Création du message

    premier = route[0] # Sélection du premier routeur
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM) # Nouveau socket
        s.connect((premier[1], premier[2])) # Connexion au routeur
        self.envoyer_ligne(s, paquet) # Envoi du message
    except Exception as e:
        print("[CLIENT] Erreur envoi", e) # Gestion des erreurs
    finally:
        s.close() # Fermeture du socket
```

-> **def lancer**

Cette méthode démarre le client dans son ensemble. Elle lance le serveur local de réception dans un thread séparé, puis affiche une interface interactive permettant à l'utilisateur de saisir les informations de destination et le message à envoyer. Elle constitue le cœur du fonctionnement du client.

```
# -----
# Lancement
# -----

def lancer(self):
    # Démarrage global du client
    th = threading.Thread(target=self.attendre_messages) # Thread serveur
    th.daemon = True # Mode daemon
    th.start() # Lancement du thread

    print(f"[{self.nom}] Prêt à envoyer") # Message prêt

    actif = True # Boucle principale
    while actif:
        print("\nNouvelle transmission") # Interface utilisateur
        ip = input("IP cible : ").strip() # Saisie IP
        p = int(input("Port cible : ").strip()) # Saisie port
        texte = input("Message : ") # Saisie message

        s = input("Sauts (vide = défaut) : ").strip() # Saisie sauts
        sauts = int(s) if s.isdigit() else self.sauts_default # Validation sauts

        self.envoyer(ip, p, texte, sauts) # Envoi du message
```

-> `__main__`

Cette partie correspond au point d'entrée du programme. Elle crée une instance du client avec les paramètres définis (nom, IP, ports, nombre de sauts par défaut) et lance son exécution.

```
# -----  
# MAIN  
# -----  
  
if __name__ == "__main__":  
    client = TerminalClient(  
        nom="A",  
        ip="127.0.0.1",  
        port=6000,  
        master_ip="127.0.0.1",  
        master_port=5000,  
        sauts_defaut=2  
    )  
  
    client.lancer()  
  
#python3 clientX.py = permet de lancer un client en fonction du choix du programme
```

Attention : Le nom et le port du Client varient en fonction du Client.

Exemple : Pour Client2

```
# -----  
# MAIN  
# -----  
  
if __name__ == "__main__":  
    client = TerminalClient(  
        nom="B",  
        ip="127.0.0.1",  
        port=6001,  
        master_ip="127.0.0.1",  
        master_port=5000,  
        sauts_defaut=2  
    )  
  
    client.lancer()  
  
#python3 clientX.py = permet de lancer un client en fonction du choix du programme
```

L'adresse 127.0.0.1 permet de réaliser des tests en local, tout les programmes communiquent en utilisant cette adresse.

Création du code pour le(s) Routeur(s)

Recherches

Pour comprendre le fonctionnement entre les différents éléments de la maquette, il m'a fallu réaliser quelque recherches sur internet, notamment sur Wikipédia ainsi que d'autres sites permettant de mettre en place les différentes parties du programme.

Évolution du code

Tout d'abord, nous avons réalisé un schéma afin de mieux comprendre le fonctionnement. Après plusieurs croquis et brouillons des différents codes, j'ai pu réalisé un premier lien entre le master, le routeur et le client.

Une fois le lien établi, j'ai réalisé deux autres routeurs pour au total en avoir minimum 3 puis un autre client pour en avoir minimum deux. Durant la réalisation des programme, chaque lignes à été expliqué par des « Docstring »

Les bibliothèques utilisées

Dans deux bibliothèques python ont été utilisées , la bibliothèque « `socket` » qui permet de réaliser la communication réseau entre les différents programmes et la bibliothèque « `threading` » qui permet de gérer plusieurs connexions en parallèle

```
import socket          # Module pour la communication réseau (TCP)
import threading       # Module pour gérer plusieurs connexions en parallèle
```

La classe « NoeudIntermediaire »

-> **def __init__**

Cette méthode initialise le routeur. Elle définit son nom (ou identifiant), son adresse IP locale et son port d'écoute, ainsi que l'adresse et le port du serveur master. Elle crée également le socket serveur du routeur, qui lui permettra de recevoir des connexions entrantes en provenance des clients ou d'autres routeurs.

```
class NoeudIntermediaire:          # Routeur renommé
    def __init__(self, nom, ip_locale, port_locale, ip_master, port_master):
        self.nom = nom             # Nom ou ID du routeur
        self.ip = ip_locale        # IP du routeur
        self.port = port_locale    # Port du routeur
        self.master_ip = ip_master # IP du serveur master
        self.master_port = port_master # Port du master

        self.sock_serveur = socket.socket(socket.AF_INET, socket.SOCK_STREAM) # Création socket TCP
        self.sock_serveur.bind((self.ip, self.port)) # Bind IP et port
        self.sock_serveur.listen(5) # Mise en écoute
```

-> **def ecrire_ligne**

Cette méthode permet d'envoyer un message texte sur un socket. Elle vérifie que le message se termine par un saut de ligne afin de respecter le protocole de communication, puis encode et envoie les données vers le destinataire.

```
# -----
# outils pour les sockets
# -----

def ecrire_ligne(self, s, texte):
    if not texte.endswith("\n"):          # Ajoute \n si nécessaire
        texte += "\n"
    s.sendall(texte.encode("utf-8"))      # Envoi du texte encodé
```

-> **def lire_ligne**

Cette méthode lit un message complet depuis un socket réseau. Elle reçoit les données octet par octet jusqu'à détecter un saut de ligne ou une fermeture de connexion. Le message est ensuite reconstruit sous forme de chaîne de caractères et retourné.

```
def lire_ligne(self, s):
    buffer = []                          # Liste pour accumuler les octets
    while True:                          # Boucle jusqu'à \n ou fermeture
        octet = s.recv(1)                # Lecture octet par octet
        if not octet:                    # Si socket fermé
            return None if not buffer else "".join(buffer) # Retourne None ou le message
        if octet == b"\n":                # Fin de ligne détectée
            break
        buffer.append(octet.decode())     # Ajoute l'octet décodé
    return "".join(buffer)                # Retourne la ligne complète
```

-> **def inscription_master**

Cette méthode permet au routeur de s'enregistrer auprès du serveur master. Elle établit une connexion avec le master, envoie un message contenant l'identifiant, l'adresse IP et le port du routeur, puis lit et affiche la réponse du master. Cette étape est indispensable pour que le routeur puisse être utilisé dans les chemins fournis aux clients.

```
# -----  
# Enregistrements au master  
# -----  
  
def inscription_master(self):  
    try:  
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM) # Socket temporaire  
        s.connect((self.master_ip, self.master_port))          # Connexion au master  
  
        message = f"ROUTER_REGISTER;{self.nom};{self.ip};{self.port}" # Message d'enregistrement  
        self.ecrire_ligne(s, message)                                # Envoi du message  
  
        resp = self.lire_ligne(s)                                    # Lecture de la réponse  
        print(f"[{self.nom}] Master a répondu : {resp}")          # Affichage  
  
        s.close()                                                  # Fermeture du socket  
    except Exception as e:  
        print(f"[{self.nom}] Erreur inscription : {e}")            # Gestion erreur
```

-> **def boucle_ecoute**

Cette méthode met le routeur en mode écoute permanente. Elle attend les connexions entrantes sur le socket serveur et crée un thread pour chaque connexion afin de traiter plusieurs messages simultanément sans bloquer le routeur.

```
# -----  
# Boucle d'écoute  
# -----  
  
def boucle_ecoute(self):  
    print(f"[{self.nom}] Routeur actif sur {self.ip}:{self.port}") # Message état  
    continuer = True  
    while continuer:                                              # Boucle principale  
        connexion, origine = self.sock_serveur.accept()          # Accepte nouvelle connexion  
        t = threading.Thread(  
            target=self.traiter_connexion,                        # Fonction de traitement  
            args=(connexion, origine)                             # Arguments  
        )  
        t.daemon = True                                          # Thread daemon  
        t.start()                                                # Démarrage thread
```


-> `def traiter_connexion`

Cette méthode traite un message reçu par le routeur. Elle analyse la commande contenue dans le message et décide de l'action à effectuer : soit transmettre le message vers le routeur suivant, soit livrer le message final au client destinataire. Elle assure également la gestion des erreurs et la fermeture de la connexion.

```
i def traiter_connexion(self, sock, origine):
    try:
        msg = self.lire_ligne(sock)          # Lecture message
        if not msg:                          # Si vide
            return
        print(f"[{self.nom}] Message reçu : {msg}") # Affichage

        elements = msg.split(";", 3)         # Découpe max 4 parties
        cmd = elements[0].upper()            # Commande en majuscules

        if cmd == "NEXT":                   # Commande intermédiaire
            if len(elements) != 4:           # Vérification format
                print(f"[{self.nom}] NEXT mal formé")
                return

            _, ip_suivant, port_suivant, data = elements # Extraction info

            if ip_suivant.upper() == "NONE": # Dernier routeur logique
                print(f"[{self.nom}] Dernier routeur → DATA = {data}")
                return

            print(f"[{self.nom}] Transfert vers {ip_suivant}:{port_suivant}")
            self.transmettre(ip_suivant, int(port_suivant), data) # Renvoi

        elif cmd == "DELIVER":              # Commande finale
            if len(elements) != 4:           # Vérification format
                print(f"[{self.nom}] DELIVER mal formé")
                return
            _, ip_client, port_client, data_final = elements # Extraction infos
            print(f"[{self.nom}] Livraison finale au client {ip_client}:{port_client}")
            self.transmettre(ip_client, int(port_client), data_final) # Envoi

        else:                               # Commande inconnue
            print(f"[{self.nom}] Commande inconnue : {cmd}")

    except Exception as e:
        print(f"[{self.nom}] Erreur traitement message : {e}") # Gestion erreur
    finally:
        sock.close()                               # Fermeture socket
```

-> **def transmettre**

Cette méthode est chargée de relayer un message vers une autre machine (routeur ou client). Elle crée une nouvelle connexion réseau vers l'adresse et le port indiqués, envoie le message et ferme la connexion. Elle constitue le mécanisme central de transmission dans le routage.

```
# -----  
# Transmission  
# -----  
  
def transmettre(self, ip, port, data):  
    try:  
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM) # Nouveau socket  
        s.connect((ip, port)) # Connexion  
        if not data.endswith("\n"): # Ajout \n si absent  
            data += "\n"  
        s.sendall(data.encode("utf-8")) # Envoi du message  
        s.close() # Fermeture socket  
    except Exception as e:  
        print(f"[{self.nom}] Impossible de joindre {ip}:{port} → {e}") # Erreur
```

-> **def demarrer**

Cette méthode démarre le routeur dans son ensemble. Elle commence par enregistrer le routeur auprès du master, puis lance la boucle d'écoute dans un thread séparé. Enfin, elle maintient le programme actif afin que le routeur reste opérationnel en permanence.

```
# -----  
# Démarrage  
# -----  
  
def demarrer(self):  
    self.inscription_master() # Inscription au master  
    th = threading.Thread(target=self.boucle_ecoute, daemon=True) # Thread écoute  
    th.start() # Lancement du thread  
    print(f"[{self.nom}] Routeur prêt.") # Message  
    threading.Event().wait() # Maintient le programme actif
```

-> `__main__`

Cette partie correspond au point d'entrée du programme. Elle définit les paramètres du routeur (identifiant, IP, port, informations du master), crée une instance du routeur et lance son exécution.

```
# -----  
# MAIN  
# -----  
  
if __name__ == "__main__":  
    ROUTEUR_ID = "R1"                # Nom du routeur  
    ROUTEUR_IP = "127.0.0.1"         # IP locale  
    ROUTEUR_PORT = 5001              # Port d'écoute (différent entre chaque routeurs)  
    MASTER_IP = "127.0.0.1"          # IP master  
    MASTER_PORT = 5000               # Port master  
  
    routeur = NoeudIntermediaire(    # Création instance routeur  
        nom=ROUTEUR_ID,  
        ip_locale=ROUTEUR_IP,  
        port_locale=ROUTEUR_PORT,  
        ip_master=MASTER_IP,  
        port_master=MASTER_PORT  
    )  
  
    routeur.demarrer()               # Lancement du routeur  
  
#python3 routeurX.py = permet de lancer un routeur en fonction du choix du programme
```

Attention : Le nom et le port du Routeur varient en fonction du Routeur.

Exemple : Pour Routeur2

```
# -----  
# MAIN  
# -----  
  
if __name__ == "__main__":  
    ROUTEUR_ID = "R2"                # Nom du routeur  
    ROUTEUR_IP = "127.0.0.1"         # IP locale  
    ROUTEUR_PORT = 5002              # Port d'écoute (différent entre chaque routeurs)  
    MASTER_IP = "127.0.0.1"          # IP master  
    MASTER_PORT = 5000               # Port master  
  
    routeur = NoeudIntermediaire(    # Création instance routeur  
        nom=ROUTEUR_ID,  
        ip_locale=ROUTEUR_IP,  
        port_locale=ROUTEUR_PORT,  
        ip_master=MASTER_IP,  
        port_master=MASTER_PORT  
    )  
  
    routeur.demarrer()               # Lancement du routeur  
  
#python3 routeurX.py = permet de lancer un routeur en fonction du choix du programme
```

L'adresse 127.0.0.1 permet de réaliser des tests en local, tout les programmes communiquent en utilisant cette adresse.

Compétences Acquises

Au cours de cette SAE, j'ai acquis la capacité d'automatiser l'administration système à l'aide de scripts Python pour gérer les communications entre clients, routeurs et serveur master (AC23.01). J'ai développé une application réseau client/serveur complète à partir d'un cahier des charges précis, en respectant les spécifications fonctionnelles (AC23.02, AC23.03). J'ai appris à installer et configurer des systèmes sur des machines virtuelles, ainsi qu'à administrer des environnements réseau simulés pour permettre la communication entre différents périphériques (AC23.04). J'ai également travaillé sur l'accès et la manipulation de données à distance via des sockets et des messages structurés (AC23.05).

Dans ce projet, j'ai su écouter les besoins du client pour définir les fonctionnalités et les flux de communication (CE3.01), tout en documentant rigoureusement le travail réalisé, notamment le code et les tests (CE3.02). J'ai utilisé les outils numériques et logiciels comme VirtualBox et Python à bon escient (CE3.03), et choisi des technologies adaptées au développement d'une application réseau (CE3.04). Enfin, j'ai intégré les problématiques de sécurité de base, en séparant les réseaux et en protégeant les flux de communication entre clients et routeurs (CE3.05).

Conclusion

En conclusion, cette SAE m'a permis de consolider de nombreuses compétences techniques et méthodologiques. J'ai appris à développer une application réseau client/serveur complète, en respectant un cahier des charges précis et en intégrant la communication entre clients, routeurs et serveur master. J'ai acquis des savoir-faire en administration système, en configurant des machines virtuelles Linux et Windows et en automatisant certaines tâches grâce à des scripts Python.

Le projet m'a également permis de maîtriser l'utilisation des protocoles réseau et de gérer des flux de données sécurisés entre différents éléments. J'ai appris à documenter mon travail et à structurer le code pour qu'il soit compréhensible et maintenable. J'ai développé ma capacité à écouter les besoins du client, à adapter les fonctionnalités et à choisir les outils de développement les plus adaptés.

J'ai pris en compte les problématiques de sécurité, en configurant correctement les réseaux et en assurant la fiabilité des communications. Cette SAE m'a donc offert une expérience complète, combinant programmation, réseau, gestion de données et méthodologie professionnelle, tout en renforçant mon autonomie et mon sens de l'organisation.

Enfin, en travaillant sur cette SAE en groupe, j'ai acquis une réelle cohésion d'équipe et appris à répartir les tâches efficacement. Nous avons su communiquer et collaborer pour résoudre les problèmes rencontrés. Cette expérience nous a permis de combiner nos compétences et de progresser ensemble, tout en respectant le cahier des charges et les contraintes du projet.